嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑　楠

sn@uestc.edu.cn

2026年3月

第四部分

编写内核

——任务管理

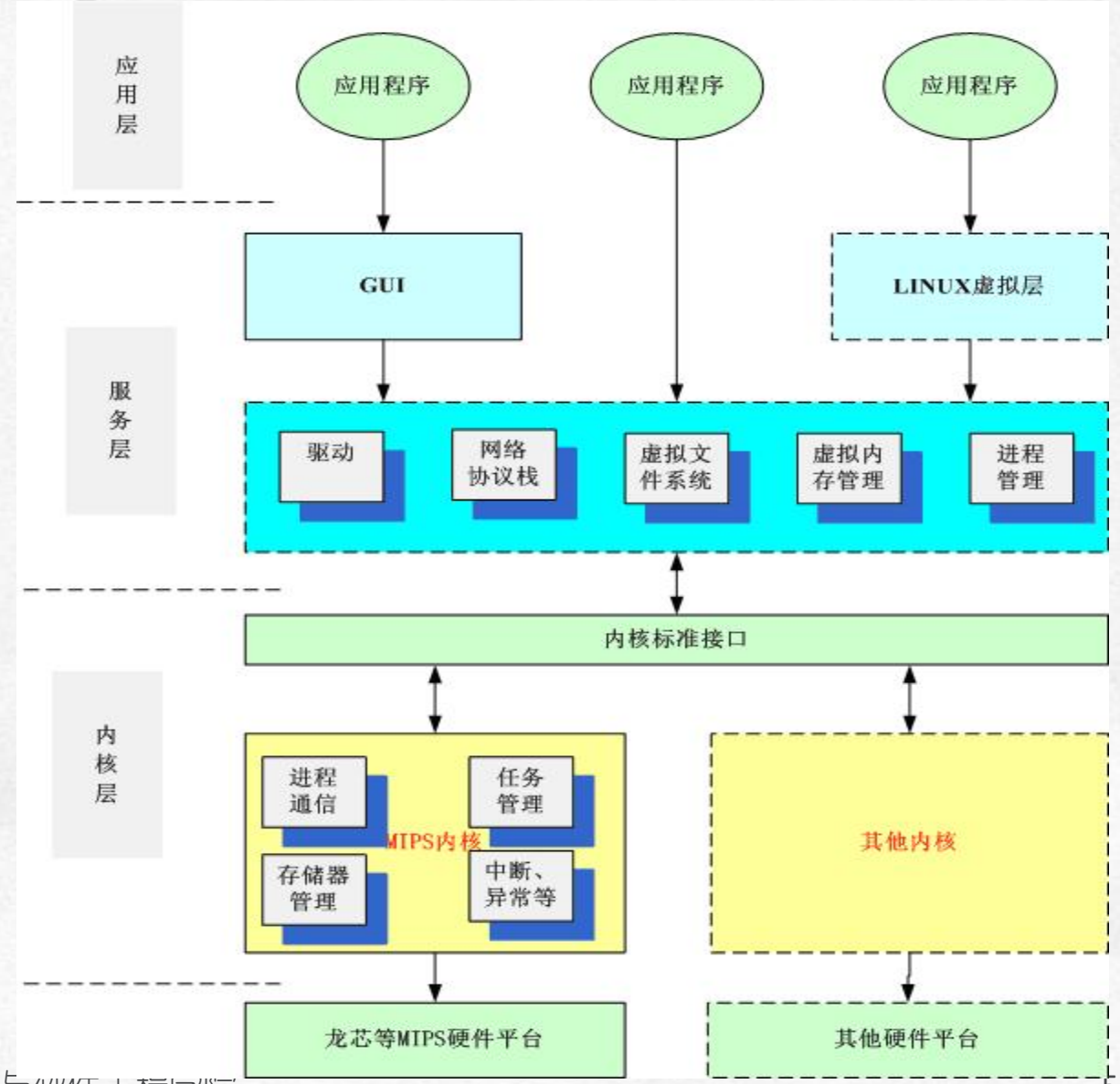
核心内容

- 嵌入式内核的基本构成
- 任务管理
- 事件处理机制
- 内存管理机制
- 任务交互机制

RTOS内核
基本构成

RTOS内核

- RTOS　对RTS，管理和控制硬件与软件资源
- RTOS内核　负责管理任务、内存、设备、文件、网络、安全、…
- 微内核　负责管理任务、进程间通信、内存、中断与时钟
- 微小内核　内核很小；但整体RTOS并非微内核结构



进程、线程 、任务

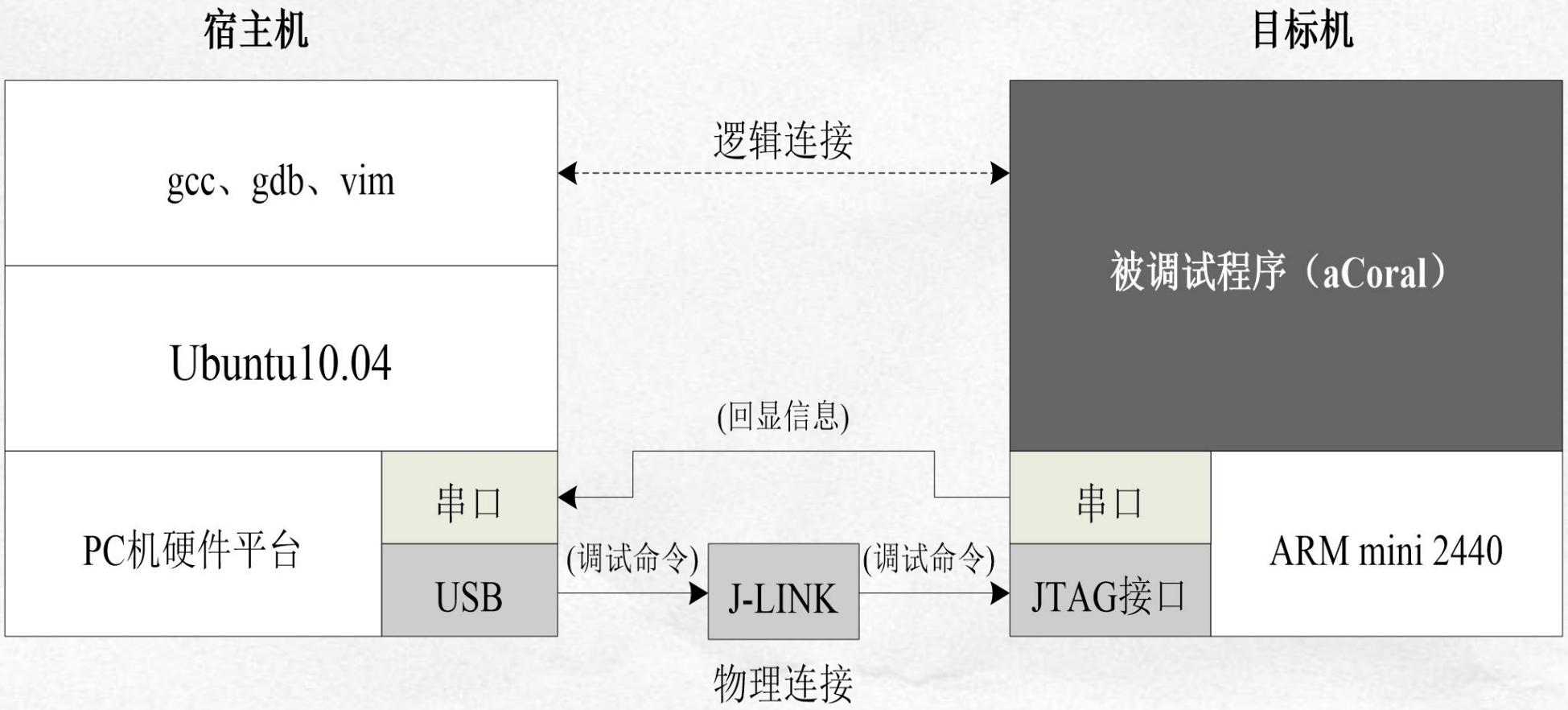
- **进程**　系统进行资源分配和~~调度~~的基本单位
- **线程**　进程内可并发执行的程序段 —— 资源共享，调度的基本单元
- **任务**　程序运行的实体，**调度的基本单元**
- **核心**　RTOS中，依据任务划分的粒度，既可为进程，也可为线程
- **本课程**　aCoral，**任务即线程**

搭建aCoral
交叉开发环境

aCoral

- a small Coral，一个开源、支持多核的微内核RTOS
- 下载地址　www.acoral.org
- 基本模块
  - 内核　电子科大嵌入式实时团队编写
  - 文件系统　来自周立功文件系统
  - 轻型TCP/IP　来自LWIP
  - GUI　来自嵌入式Linux图形系统LGUI
  - 简单应用服务　网页服务器、Telent服务、文件操作、GUI图形、测试、…
- 内核大小　最小配置7k；待GUI等后300k；
- 处理器支持　ARM系列

结构示意

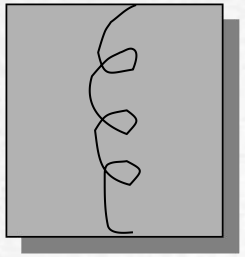

宿主机
目标机
gcc、gdb、vim
逻辑连接
被调试程序（aCoral）
Ubuntu10.04
(回显信息)
串口
串口
PC机硬件平台
(调试命令)
(调试命令)
ARM mini 2440
USB
J-LINK
JTAG接口
物理连接

安装工具链

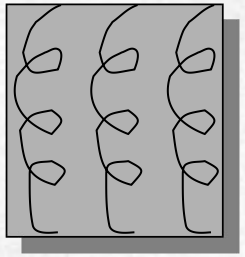
- 安装ubuntu　3G以上的磁盘空间(建议值是10G以上)
- 安装交叉开发工具链
  - 工具类别　gcc、gdb、vim（编辑）、ncurses（图形互动库）、...
  - 编译工具　arm-linux-gcc-3.4.5-glibc-2.3.6.tar.bz2
  - .....
- 构建aCoral项目　aCoral项目相关的文件夹
  - kernel(内核)、HAL(硬件抽象层)、driver(驱动)、plugin(项目扩展插件)、lib(库)、user(用户程序)、config(配置)、tools(工具)、test(测试)、...
  - 每个程序文件　src(源码)、include(头文件)

任务管理

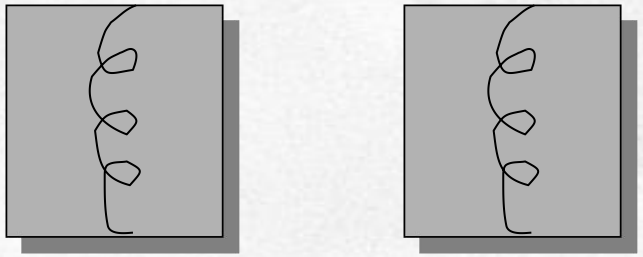
应用系统的任务构成



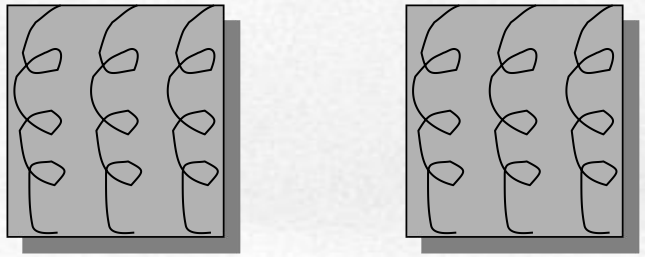
单进程/单任务模型



单进程/多任务模型

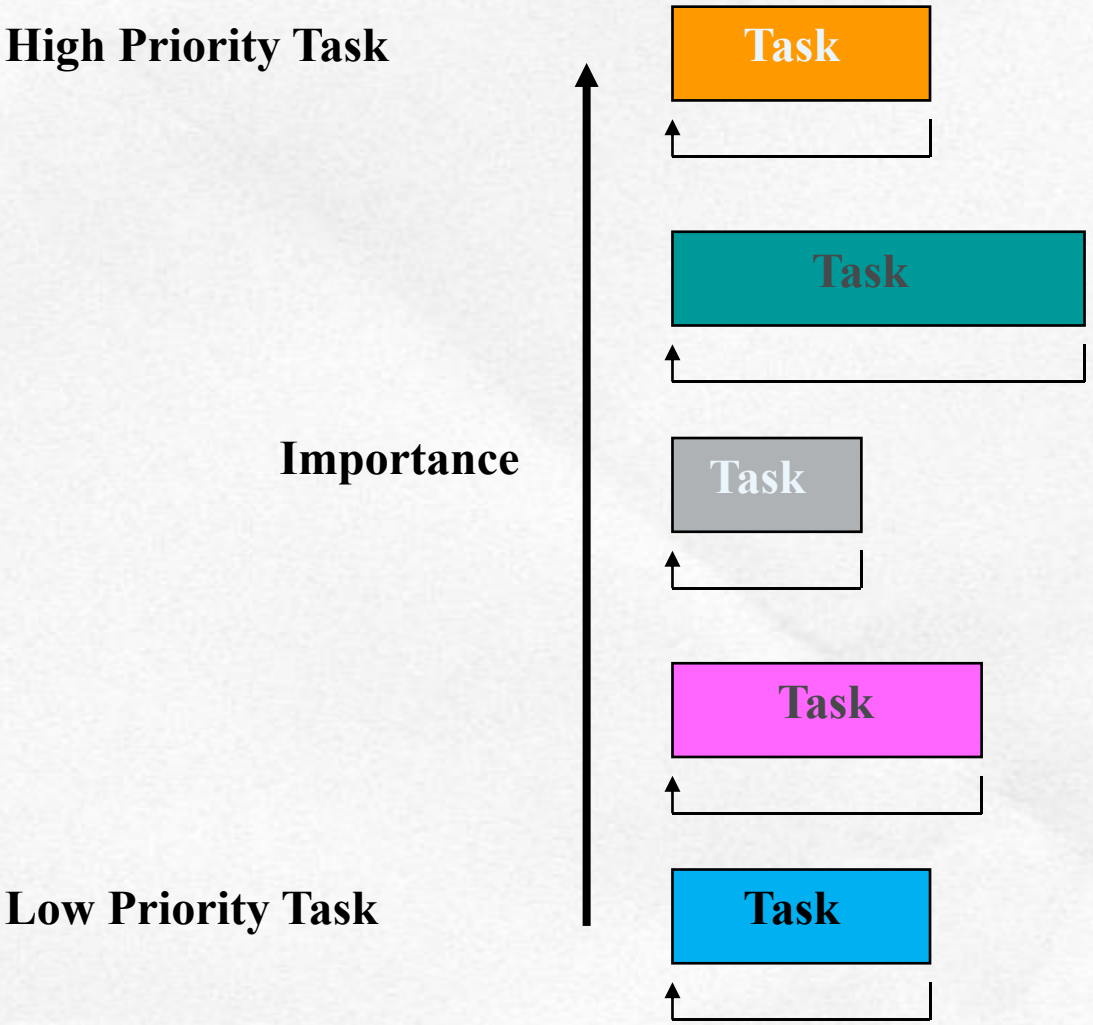


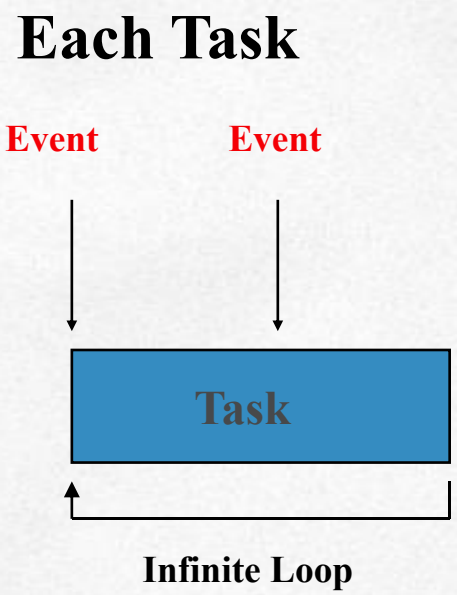
多进程/单任务模型

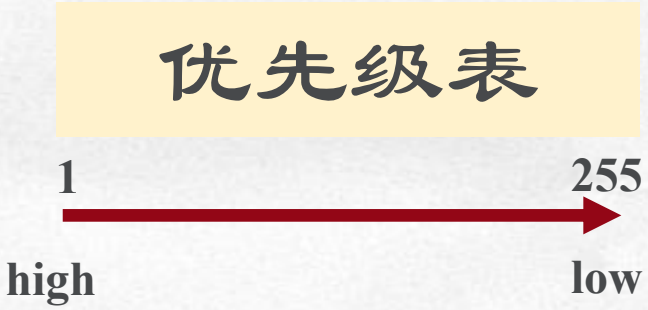


多进程/多任务模型

将应用分解为任务


High Priority Task
Task
Task
Importance
Task
Task
Low Priority Task
Task


Each Task
Event
Event
Task
Infinite Loop


优先级表
1
255
high
low

描述任务（线程）

```
// 线程结构
ACORAL_COMM_THREAD  test3(acoral_u32 timer){
        while {
                acoral_delay_self(timer)
        }
}

// 创建32个线程：代码相同，但环境不同
void test_delay_init() {
        for (i=0; i<34; i++) {
                acoral_print("%d\n", i);
                id=acoral_create_thread(test3, 255, 500+i*10, "delay", i+1, -1);
        }
}
```

任务与程序的区别

- 任务能真实地描述工作内容的**并发性**，而程序不能
- **程序是任务的组成部分**：任务还包括数据、堆栈及其上下文环境等（TCB）
- 程序是**静态的**，任务是**动态的**
- 任务有**生命周期**：诞生 → 消亡，是短暂的；而程序相对长久永存
- 一个程序可对应多个任务；反之亦然
- 任务具有创建等功能；而程序没有

任务类型

非实时任务 — 实时任务

非关键任务 — 关键任务

硬实时任务 — 软实时任务

- 实时任务
  - 周期任务：固定时间间隔内必须执行一次的任务
  - 突发任务：任务实例的到达时刻不是严格周期的的任务 —— 一般可转为周期任务
  - 非周期任务：随机到达系统的任务

硬嵌入式实时系统：实时确保？

实时调度算法

- 如RM、EDF

ERTOS内核的设计

- 如TCB的设计

理论与工程背景结合

- 如前端人脸识别

任务控制块（TCB）

- **OS的TCB**：任务相关信息
  - 任务描述信息
  - 任务调度信息
  - 任务控制信息
  - 处理器现场保护信息
- **ERTOS的TCB**：？？

任务控制块（续）

- TCB结构

任务名
任务号
任务当前状态
任务优先级
任务上下文 （寄存器、堆栈、指针等）
任务周期
任务执行时间
任务截止时间
。 。 。

信息与

- aCoral的TCB

```
typedef struct {
        acoral_res_t  res                          // 资源控制
#ifdef  CFG_CMP
        acoral_spinlock_t  move_lock;              // 支持多核
#endif
        acoral_u8         state;                   // 线程状态
        acoral_u8         prio;                    // 优先级
        acoral_8          CPU;                     // 所属CPU
        acoral_u32        CPU_mask;
        acoral_u8         policy;                  // 调度策略
        acoral_list_t     ready;                   // 就绪队列
        acoral_list_t     timeout;                 // 超时队列
        acoral_list_t     waiting;                 // 延时队列
        acoral_list_t     global_list;             // 全局线程
        acoral_evt_t      *evt;                    // 占用事件
        acoral_u32        *stack;                  // 所用堆栈
        acoral_u32        *stack_buttom;           // 栈底
        acoral_u32        stack_size;              // 栈大小
        acoral_32         delay;                   // 延迟ticks
        acoral_u32        slice;                   // 轮询时间片
        acoral_char       *name;                   // 名称
        acoral_id         console_id;              // 控制台id号
        void*             private_data             // 备用
        void*             data;                    // 备用
} acoral_thread_t;
```

任务控制块（2）

```
……
        acoral_spinlock_t move_lock;  // 支持多核
……
        acoral_u8
        acoral_u8
        acoral_u8
……
        acoral_u32
        acoral_char
……
```

```
……
        acoral_list_t   ready;          // 就绪队列
        acoral_list_t   timeout;        // 超时队列
        acoral_list_t   waiting;        // 延时队列
        acoral_list_t   global_list;    // 全局线程
……
        acoral_u32  *stack;             // 所用堆栈
        acoral_u32  *stack_buttom;      // 栈底
        acoral_u32  stack_size;         // 栈大小
        acoral_32           delay;      // 延迟ticks
……;
```

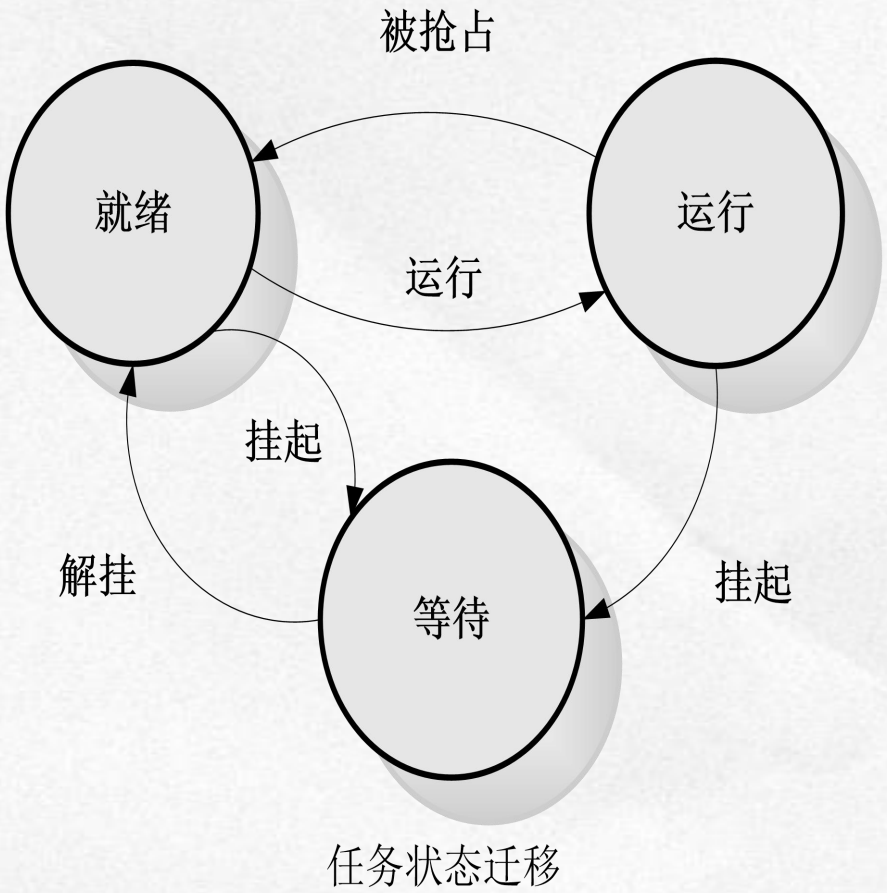
资源控制块

- aCoral中的资源：**六种，支持多核共享**
  - 线程型（TCB）
  - 事件型
  - 时钟型
  - 驱动型
  - GUI型
  - 用户使用型

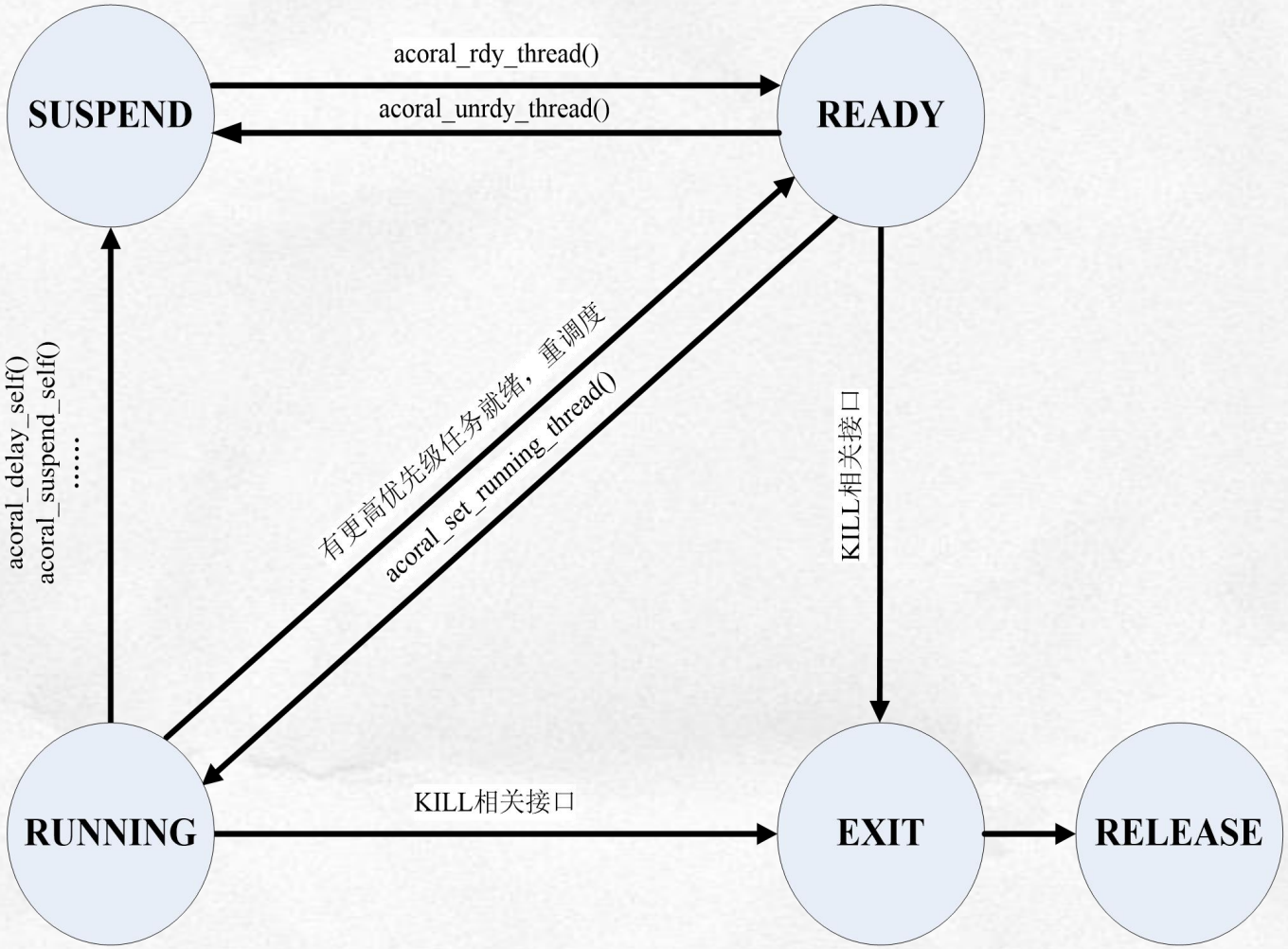
```
typedef struct {
  acoral_u32    type;                          // 资源类型
  acoral_u32    size;                          // 资源结构体大小
  acoral_u32    num_per_pool;                  // 资源池中对象数
  acoral_u32    num;                           // 资源池数
  acoral_u32    max_pools;
  acoral_list_t  *free_pools,*pools,list[2];   // 空闲池链表
  acoral_res_api_t        *api;                // 资源操作API入口
#ifdef CFG_CMP
  acoral_spinlock_t  lock;                     // 多核共享锁
#endif
  acoral_u8      *name;                        // 资源名称
}acoral_pool_ctrl_t;
```

任务状态切换

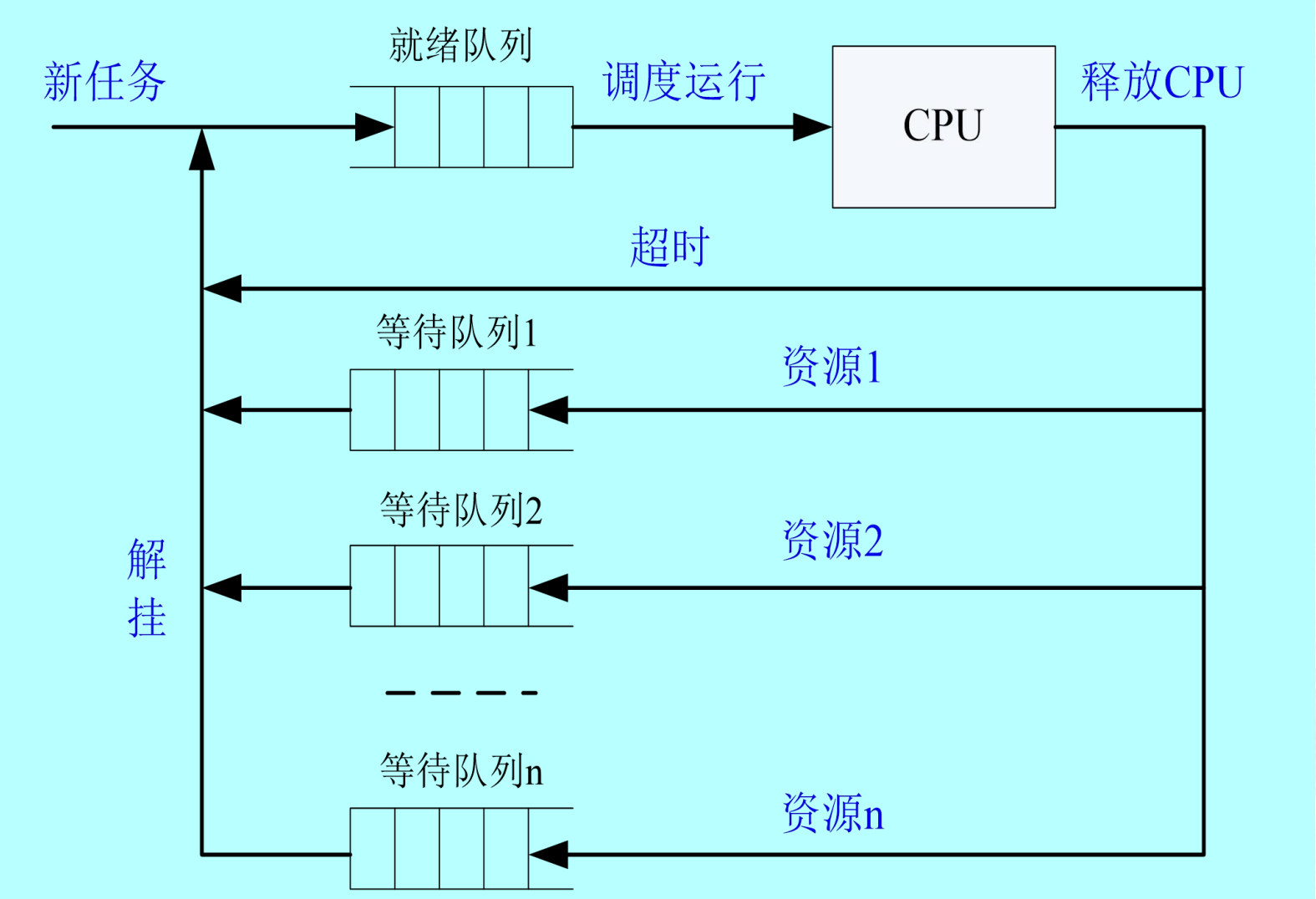
- 基本状态切换



- aCoral的任务状态切换



任务队列



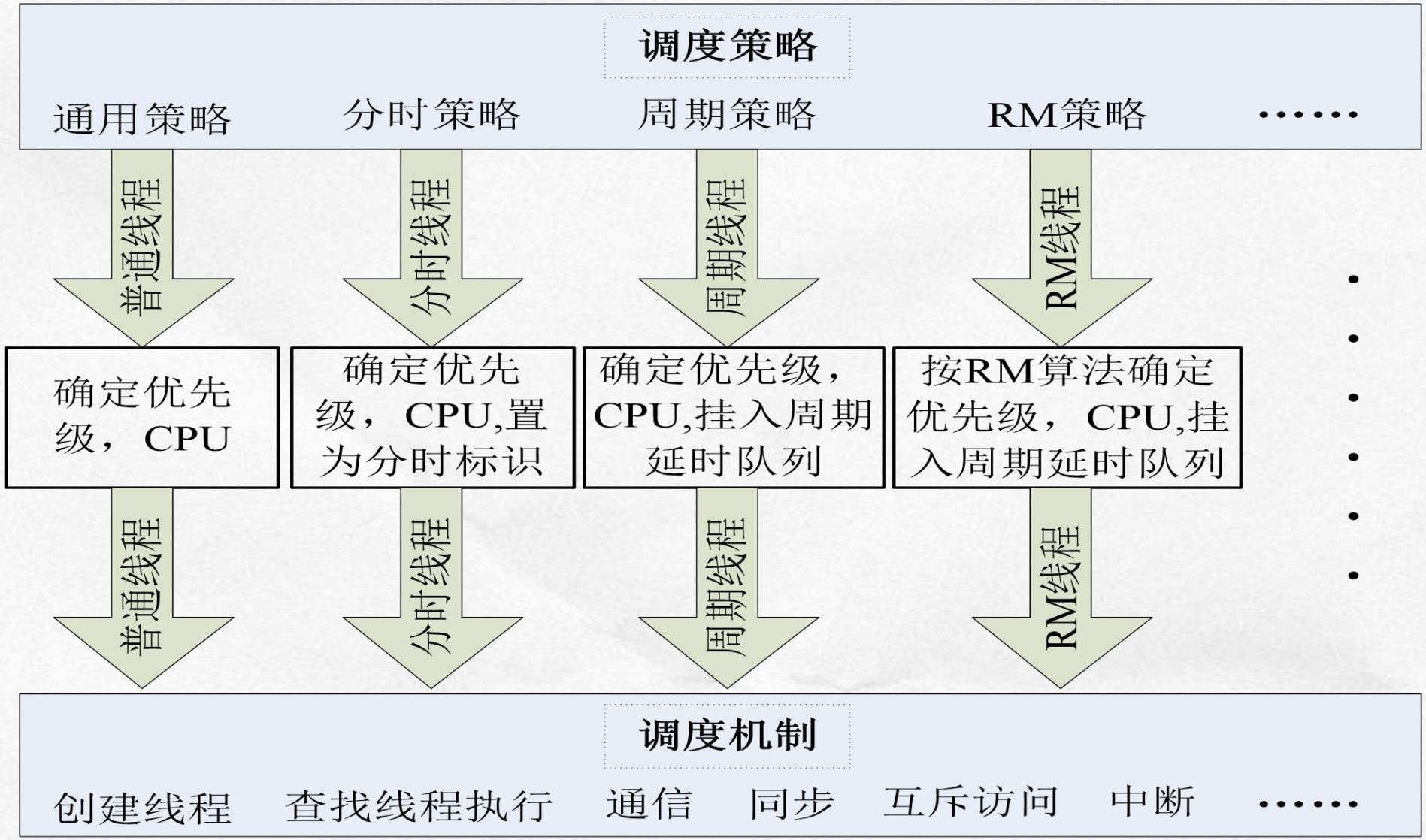
单就绪多等待队列结构

任务调度策略

确定任务执行顺序的基本原则

- **静态调度表驱动**　类似火车时刻表；静态，开销小，可预测，实时保障强；不灵活
- **基于优先权的调度**　使高优先级任务优先处理；优先级可静态赋予，或动态生成
  - 可抢占的调度　高优先级到达即开始执行
  - 非抢占的调度　当前任务完成，再执行就绪队列中的高优先级任务
- **时间片的调度**　为每个任务分配每次执行的时间片，循环执行或依某种规则执行

aCoral的任务调度策略



aCoral调度策略对应的数据结构

- 不同调度策略需要不同的数据结构
- 普通调度策略

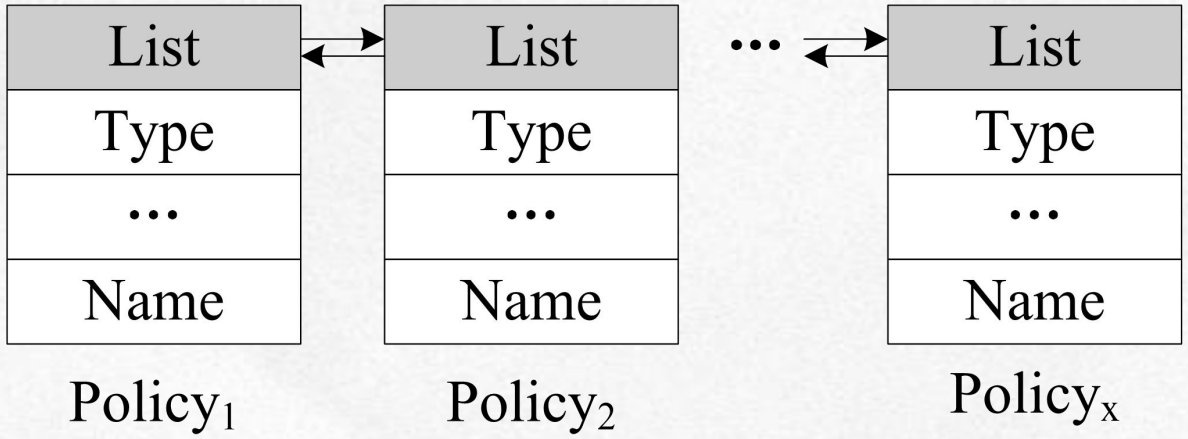
```
typedef struct{
        acoral_8 CPU;
        acoral_u8 prio;
}acoral_comm_policy_data_t;
```

- 周期型调度策略

```
typedef struct{
        acoral_8 CPU;
        acoral_u8 prio;
        acoral_time time;                    // 任务的周期
}acoral_period_policy_data_t;
```

aCoral的调度策略控制表

- 调度策略控制块acoral_sched_policy_t



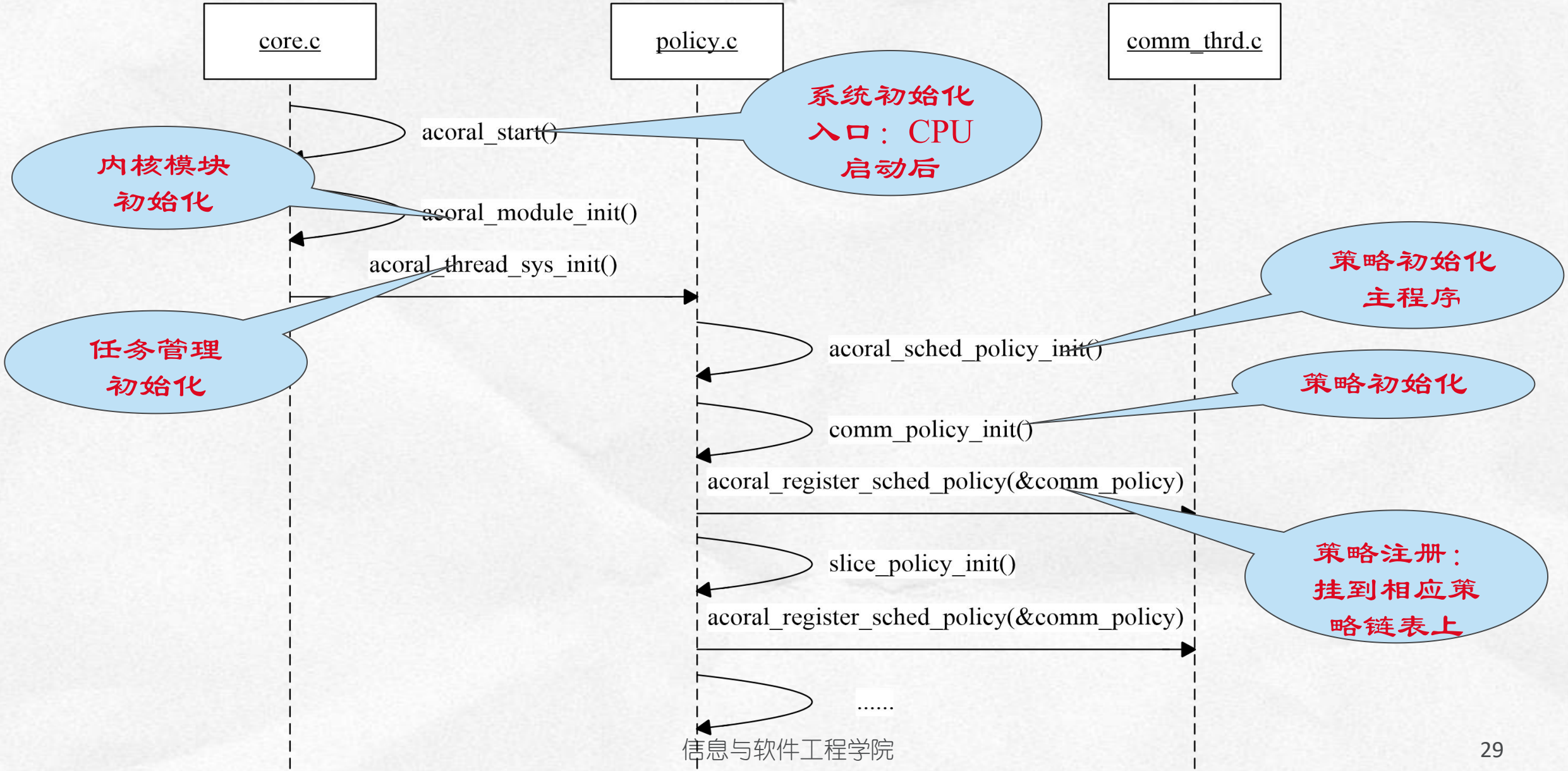
- 策略类型
  - ACORAL_SCHED_POLICY_COMM,
  - ACORAL_SCHED_POLICY_SLICE,
  - ACORAL_SCHED_POLICY_PERIOD,
  - ACORAL_SCHED_POLICY_RM,
  - ACORAL_SCHED_POLICY_POSIX

aCoral调度策略对应的数据结构（2）

- RM（速率单调）调度策略

```
typedef struct{
        acoral_8 CPU;
        acoral_u8 prio;
        acoral_u32 t;                    // 任务周期
        acoral_u32 e;                    // 任务执行时间
}acoral_rm_policy_data_t;
```

aCoral调度策略的初始化及注册



aCoral调度策略的初始化及注册（2）

- **示例**：普通任务调度策略初始化及注册

```
void comm_policy_init(){
        comm_policy.type=ACORAL_SCHED_POLICY_COMM;
        //绑定策略初始化函数：普通策略
        comm_policy.policy_thread_init=comm_policy_thread_init;
        comm_policy.policy_thread_release=NULL;
        comm_policy.delay_deal=NULL;
        comm_policy.name="comm";
        // 策略注册
        acoral_register_sched_policy(&comm_policy);
}
```

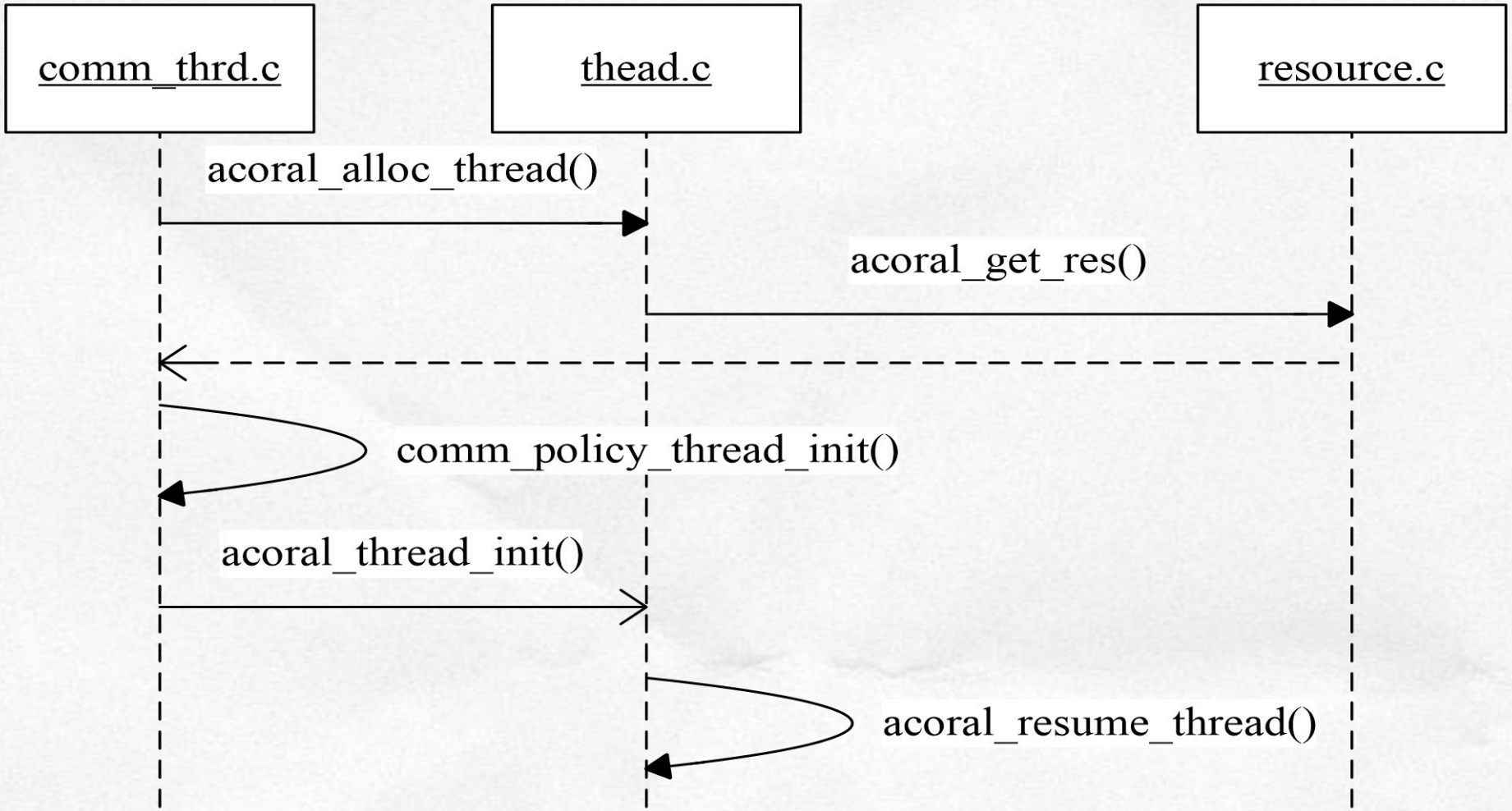
任务管理的基本功能

- **创建任务**　分配控制块、初始化参数、 创建
- **调度任务**　已具备多个任务并发执行的环境，选择高优先级任务执行
- **退出任务**　任务执行完成后，退出；释放资源
- **挂起任务**　需要请求某一被占用资源时，任务挂起
- **恢复任务**　挂起的逆过程 —— 资源可用时，恢复任务
- **改变任务优先级**　动态改变任务优先级
- **调度任务的时间处理**　维护任务所需的时间参数：**周期、时间片、…**

创建任务

- 创建任务的过程
  - 分配任务空间：acoral_get_res()，资源空间分配
  - 策略初始化：acoral_policy_thread_init()，选择策略，初始任务参数
  - 任务初始化：acoral_thread_init()，初始堆栈空间、任务环境（寄存器）
  - 挂就绪队列：acoral_resume_thread()，按一定的规则挂入就绪队列，等待任务启动
  - 启动调度：**acoral_sched()**，选择当前执行任务

创建任务：普通任务创建过程



创建任务 —— 核心部分

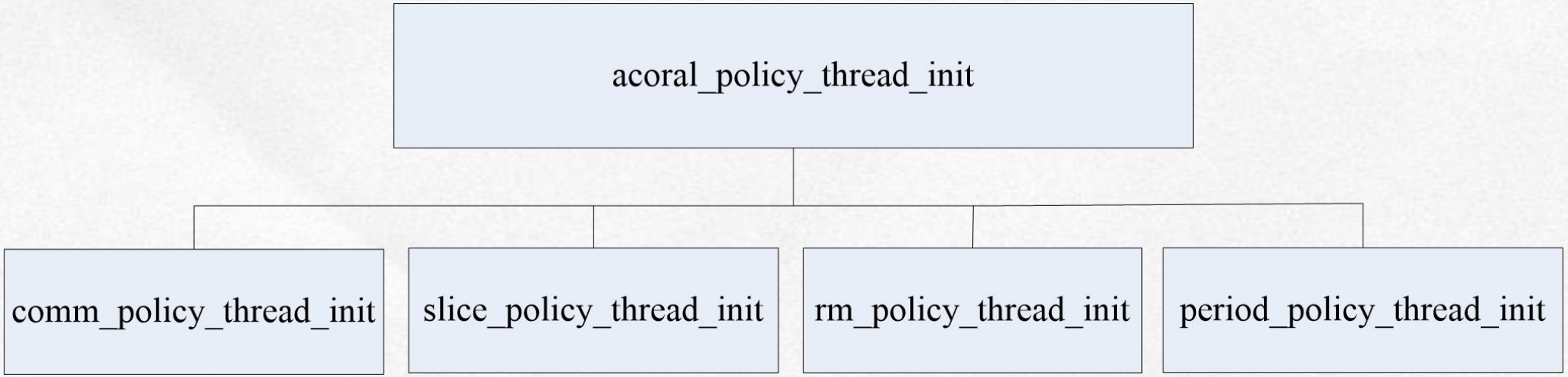
- **示例：** 创建一个周期型任务

策略控制块

```
acoral_period_policy_data_t data;
// TCB中任务参数初始化
data.CPU=0;
data.prio=25;
data.time=1000;
// 创建
acoral_create_thread_ext(test,  ACORAL_PRINT_STACK_SIZE, 0,
                         NULL, NULL, ACORAL_SCHED_POLICY_PERIOD,
                         &data);
```

创建任务 —— 策略初始化

- 分类型处理　　acoral_policy_thread_init()
- 完成TCB参数初始赋值，调用acoral_thread_init()



创建任务 —— 策略初始化（2）

- 普通策略初始化

将策略控制块的值赋给TCB

```
acoral_id comm_policy_thread_init(acoral_thread_t *thread,
        void (*route)(void *args), void *args,void *data){
    acoral_sr CPU_sr;
    acoral_u32 prio;
    acoral_comm_policy_data_t  *policy_data;
    policy_data=(acoral_comm_policy_data_t *)data;
    thread->CPU=policy_data->CPU;
    prio=policy_data->prio;
    if(policy_data->prio_type==ACORAL_BASE_PRIO){
```

创建任务 —— 策略初始化（3）

```
if(policy_data->prio_type==ACORAL_BASE_PRIO){
    prio+=ACORAL_BASE_PRIO_MIN;
    if(prio>=ACORAL_BASE_PRIO_MAX)
        prio=ACORAL_BASE_PRIO_MAX-1;
}
thread->prio=prio;
if(acoral_thread_init(thread,route,acoral_thread_exit,args)!=0){
```

创建任务 —— 策略初始化（4）

```
if(acoral_thread_init(thread,route,acoral_thread_exit,args)!=0){
    acoral_printer("No thread stack:%s\n",thread->name);
    HAL_ENTER_CRITICAL();
    acoral_release_res((acoral_res_t *)thread);
    HAL_EXIT_CRITICAL();
    return -1;
}
acoral_resume_thread(thread);            //将线程就绪，并重新调度
return thread->res.id;
}
```

任务初始化：TCB其他参数赋值

堆栈分配失败：释放资源

挂就绪队列

创建任务 —— 任务初始化

- 堆栈相关初始化

```
acoral_err acoral_thread_init(acoral_thread_t *thread,
        void (*route)(void *args),void (*exit)(void),void *args){
    if(thread->stack_buttom==NULL){
        if(stack_size<ACORAL_MIN_STACK_SIZE)
            stack_size=ACORAL_MIN_STACK_SIZE;
        thread->stack_buttom=(acoral_u32 *)acoral_malloc(stack_size);
        if(thread->stack_buttom==NULL)
            return ACORAL_ERR_THREAD_NO_STACK;
        thread->stack_size=stack_size;
    }
```

堆栈未分配

分配不成功

创建任务 —— 任务初始化（2）

```
thread->stack=(acoral_u32 *)((acoral_8 *)thread->stack_buttom
HAL_STACK_INIT(&thread->stack,route,exit,args);
if(thread->CPU_mask==-1)
    thread->CPU_mask=0xeffffff;
if(thread->CPU<0)
    thread->CPU=acoral_get_idle_maskCPU(thread->CPU_mask);
if(thread->CPU>=HAL_MAX_CPU)
    thread->CPU=HAL_MAX_CPU-1;
thread->data=NULL;
thread->state=ACORAL_THREAD_STATE_SUSPEND;
```

运行环境初始化：各寄存器保存次序 —— 依使用的CPU

确定可用CPU

任务状态为挂起

创建任务 —— 任务初始化（3）

```
……
acoral_init_list(&thread->waiting);        // 等待队列初始化
acoral_init_list(&thread->ready);
acoral_init_list(&thread->timeout);
acoral_init_list(&thread->global_list);
acoral_spin_init(&thread->timeout.lock);
acoral_spin_init(&thread->waiting.lock);
acoral_spin_init(&thread->ready.lock);
acoral_spin_init(&thread->move_lock);
```

初始化各种队列：就绪、…

初始化队列相关自旋锁：多核

创建任务 —— 任务初始化（4）

```
    HAL_ENTER_CRITICAL();
    acoral_spin_lock(&acoral_threads_queue.head.lock);
    acoral_list_add2_tail(&thread->global_list,&acoral_threads_queue.head);
    acoral_spin_unlock(&acoral_threads_queue.head.lock);
    HAL_EXIT_CRITICAL();
    return 0;
}
```

封锁、使用、解锁

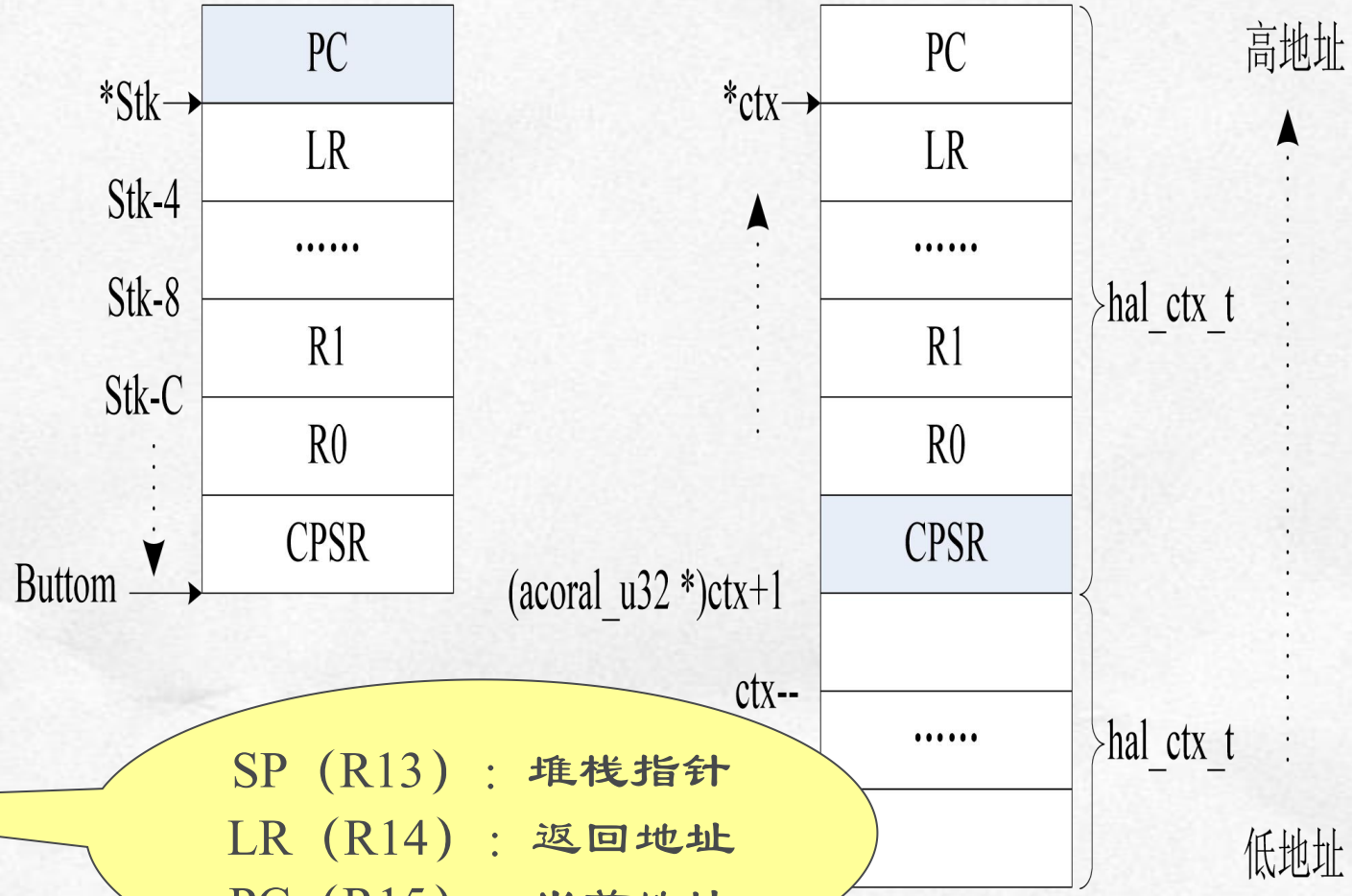
新任务加入全局队列尾

示例：ARMS3C2440的任务运行环境

- 环境数据结构：16个寄存器

```
typedef struct {
        acoral_u32 cpsr;
        acoral_u32 r0;
        acoral_u32 r1;
        acoral_u32 r2;
        acoral_u32 r3;
        acoral_u32 r4;
        acoral_u32 r5;
        acoral_u32 r6;
        acoral_u32 r7;
        acoral_u32 r8;
        acoral_u32 r9;
        acoral_u32 r10;
        acoral_u32 r11;
        acoral_u32 r12;
        acoral_u32 lr;
        acoral_u32 pc;
}hal_ctx_t;
```

- 堆栈保存次序



SP（R13）：堆栈指针
LR（R14）：返回地址
PC（R15）：当前地址

创建任务 —— 挂就绪队列

```
void acoral_resume_thread(acoral_thread_t *thread){
    acoral_sr CPU_sr;
    acoral_8 CPU;
    if(!(thread->state&ACORAL_THREAD_STATE_SUSPEND))
        return;
    ......
    /*resumed*/
    if(CPU!=acoral_current_CPU){
        /*给特定CPU发送命令*/
        acoral_ipi_cmd_send(CPU, ACORAL_IPI_THREAD_RESUME,
                thread->res.id, NULL);
        return;
    }
```

挂起状态
不调度

确定当前CPU

创建任务 —— 挂就绪队列（2）

```
    return;
}
#endif
HAL_ENTER_CRITICAL();
/*将线程挂到指定CPU的就绪队列上*/
acoral_rdyqueue_add(thread);
HAL_EXIT_CRITICAL();
acoral_sched();
}
```

重新调度

创建任务 —— 挂就绪队列（3）

```
void acoral_rdyqueue_add(acoral_thread_t *thread){// 加入就绪队列
        acoral_rdy_queue_t *rdy_queue;
        acoral_u8 CPU;
        CPU=thread->CPU;
        rdy_queue=acoral_ready_queues+CPU;
        acoral_prio_queue_add(&rdy_queue->array,thread->prio,&thread->ready);
        thread->state&=~ACORAL_THREAD_STATE_SUSPEND;
        thread->state|=ACORAL_THREAD_STATE_READY;
        thread->res.id=thread->res.id|CPU<<ACORAL_RES_CPU_BIT;
        /*设置任务调度标记为True*/
        acoral_set_need_sched(true);
}
```

找到CPU
就绪队列

添加

准备调度

创建任务 —— 特殊任务创建

- 过程相同，初始化参数不同：特殊任务参数

```
acoral_id create_thread_ext(void (*route)(void *args),acoral_u32 stack_size,void *args,acoral_char
*name,void *stack,acoral_u32 sched_policy,void *data){
        acoral_thread_t *thread;
        thread=acoral_alloc_thread();                    //返回刚分配的TCB的指针
        ......
        // 基本参数初始化
        thread->name=name;
        thread->stack_size=stack_size;
        thread->policy=sched_policy;
        // 取相应策略，初始化策略和任务
        return acoral_policy_thread_init(sched_policy,thread,route,args,data); //策略、TCB、线程函数、参数、策略数据
}
```

创建任务 —— 特殊策略初始化

- 依据策略控制块，确定策略类型 → 时间片调度策略初始化

```
acoral_id slice_policy_thread_init(acoral_thread_t *thread, ... ) {
        ......
        if(thread->policy==ACORAL_SCHED_POLICY_SLICE){
                policy_data=(acoral_slice_policy_data_t *)data;
                // 其余公共参数初始化：CPU、prio、...
                ......
                private_data=(slice_policy_data_t *)acoral_malloc2(sizeof(slice_policy_data_t));
                ......
                private_data->slice_ld=TIME_TO_TickS(policy_data->slice);
                thread->slice=private_data->slice_ld;
        }
        if(acoral_thread_init(thread,route,acoral_thread_exit,args)!=0){
                ......                          // 初始化失败处理
                return -1;
        }
        acoral_resume_thread(thread);           // 准备重新调度
        return thread->res.id;
```

时间片策略控制块

TCB分配空间

时间片换成 ticks

调度任务

- 调度任务的过程
  - 确定任务调度的时机
  - 主动调度与被动调度
  - 任务调度：判断是否可调度？执行调度过程
  - 获取最高优先级任务：确定当前哪个任务执行
  - 任务切换：当前任务挂起，高优先级任务执行

调度任务：任务调度的时机

- 即何时何地调用调度程序acoral_sched()
- **调度点** Scheduling Point，调用调度程序的具体位置
- **主要的调度点**
  - 中断服务结束时
  - 任务因等待资源而被阻塞（挂起）时
  - 周期开始或周期结束时
  - 高优先级任务达到时
  - 时间片结束时
  - 关调度等状态打开时
  - ……

调度任务：主动与被动

- **主动调度**　　任务主动调用调度函数，根据调度算法选择需要执行的任务，如**当前执行任务的时间片完**
- **被动调度**　　某个事件触发的调度，如**周期到达、高优先级任务就绪、突发事件、…**

调度任务：是否可调度

- 在合适的调度点上，是否可以直接开始调度？
- 必须的关注点
  - 调度开始标志acoral_ start_sched　任务初始化完？
  - 是否需要调度标志acoral_need_sched　当进行一些操作导致需要调度时标记
  - 是否中断执行中　类似高优先级任务正执行
  - 调度锁　禁止调度标识，类似关中断
  - ……

调度任务 —— 是否可调度（2）

```
void acoral_sched(){
        if (!acoral_need_sched)                 // 判断是否需要调度，如果不需要，则返回
                return;
        if (acoral_intr_nesting)                // 如果属于嵌套中断
                return;
        if(acoral_sched_is_lock)                // 如果调度被屏蔽(禁止调度)
                return;
        if(!acoral_start_sched)                 // 如果还没有开始调度，则返回
                return;
        // 进行简单处理后会直接或间接调用acoral_real_sched()
        HAL_SCHED_BRIDGE();
        return;
}
```

硬件抽象层处理

调度任务：直接调用

```
void acoral_sched(){
        if (!acoral_need_sched)                    // 判断是否需要调度，如果不需要，则返回
                return;
        ......
        if(!acoral_start_sched)                    // 如果还没有开始调度，则返回
                return;
        // 进行简单处理后会直接调用acoral_real_sched()
        HAL_ENTER_CRITICAL();
        acoral_real_sched();
        HAL_EXIT_CRITICAL()     return;
}
```

进入临界区

调度任务：间接调用

```
#define HAL_SCHED_BRIDGE()  hal_sched_bridge_comm()


static inline void hal_sched_bridge_comm(){
        acoral_sr CPU_sr;
        HAL_ENTER_CRITICAL();
        acoral_real_sched();
        HAL_EXIT_CRITICAL();
}
```

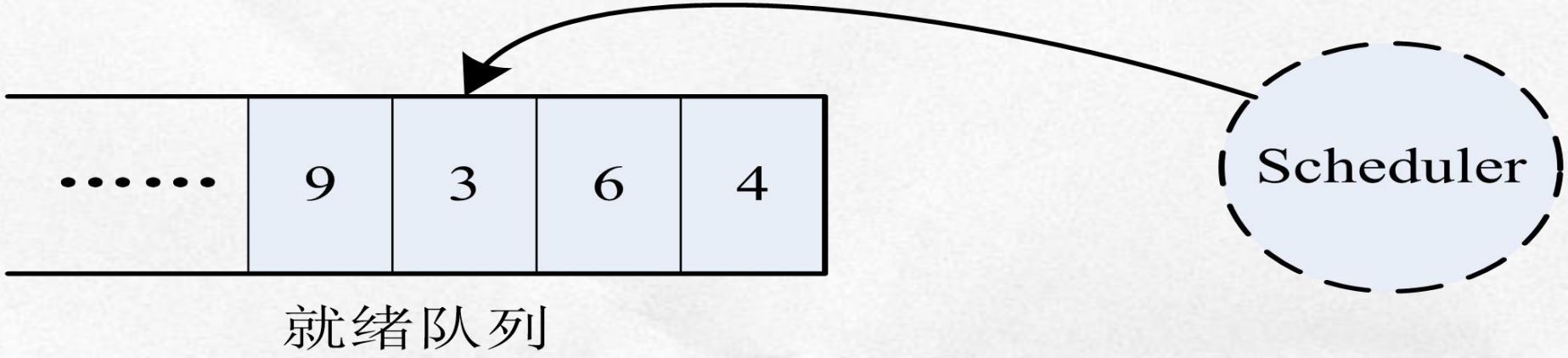
对一些类型的处理器，确保acoral_real_sched()的原子性

进入临界区

- **原子性**　执行过程中不可被打断（中断、调度）

调度任务：最高优先级任务

- acoral_select_thread();
- **从就绪队列中找出最高优先级任务**



调度任务：最高优先级任务（2）

```
void acoral_select_thread(){
        ......
        acoral_rdy_queue_t *rdy_queue;
        acoral_prio_array_t *array;
        acoral_list_t *head;
        acoral_thread_t *thread;
        acoral_queue_t *queue;
        CPU=acoral_current_CPU;                 // 针对当前CPU
        rdy_queue=acoral_ready_queues+CPU;      // 对应当前CPU的就绪队列
        array=&rdy_queue->array;
        index = acoral_get_highprio(array);     // 找出最高优先级线程的优先级
        queue = array->queue + index;
        head=&queue->head;
        thread=list_entry(head->next, acoral_thread_t, ready);
        ......
}
```

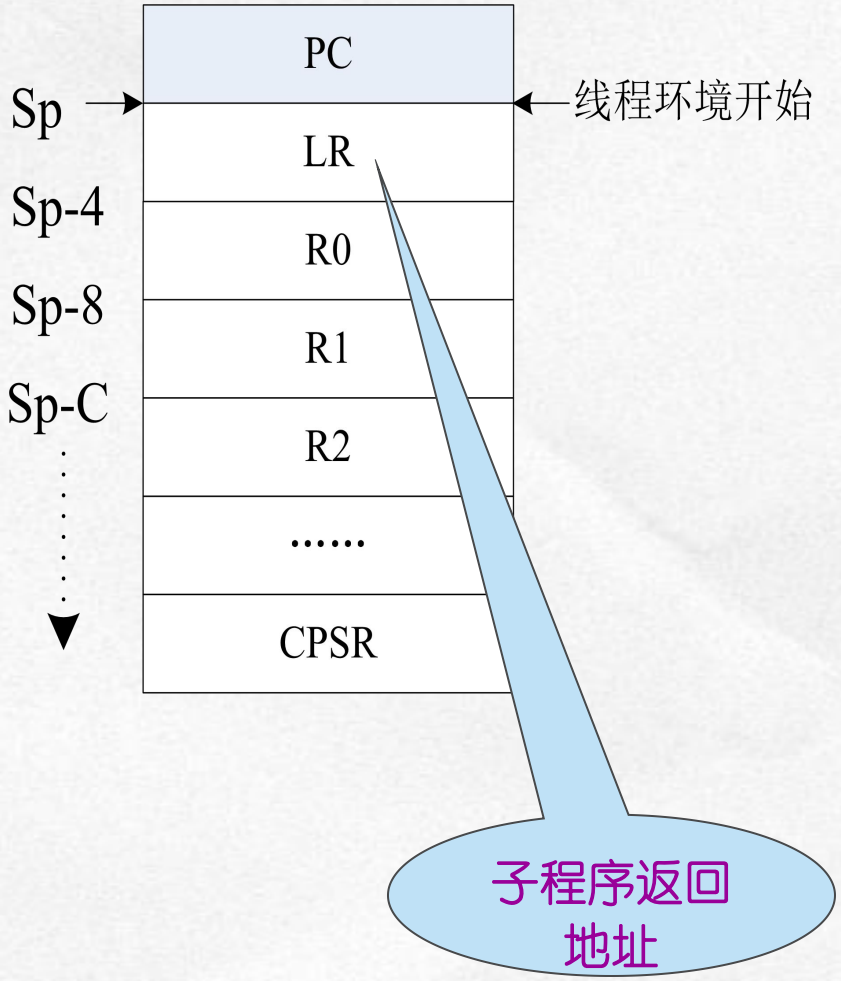
优先级数组（位图）

调度任务：最高优先级任务（3）

- **就绪队列的构造方式**　　顺序、按优先级
- **顺序**　　任务到达次序依次插入，快速；最高优先级查找用位图、二分法、…（**数据结构**）；aCoral
- **按优先级**　　任务到达时按优先级排序插入：二分法、…（**数据结构**）；查找快速；Mach kernel

调度任务：任务切换

- 任务环境信息



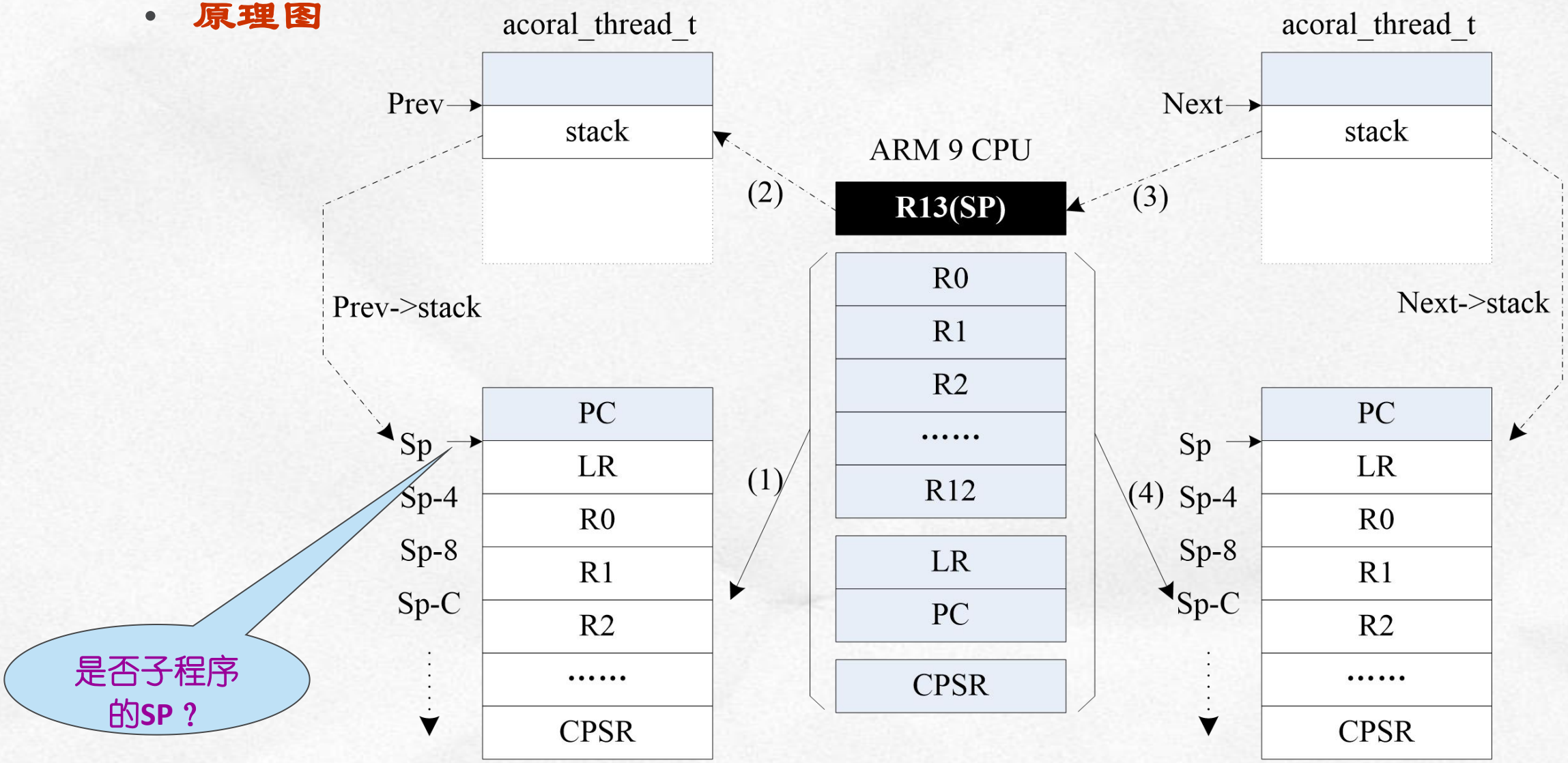
- 任务切换程序

```
HAL_CONTEXT_SWITCH:
    @保存被抢占任务现场
    stmfd   sp!,{lr}            @PC
    stmfd   sp!,{r0-r12,lr}     @LR，及r0-r12
    mrs     r4,CPSR
    stmfd   sp!,{r4}            @cpsr
    str     sp,[r0]             @栈指针prev->stack

    @恢复新任务现场
    ldr     sp,[r1]             @新上下文指针
    ldmfd   sp!, {r0}
    msr     cpsr,r0             @cpsr(不能用spsr
                                @，sys,user模式无SPSR)
    ldmfd   sp!, {r0-r12,lr,pc}     @寄存器,
```

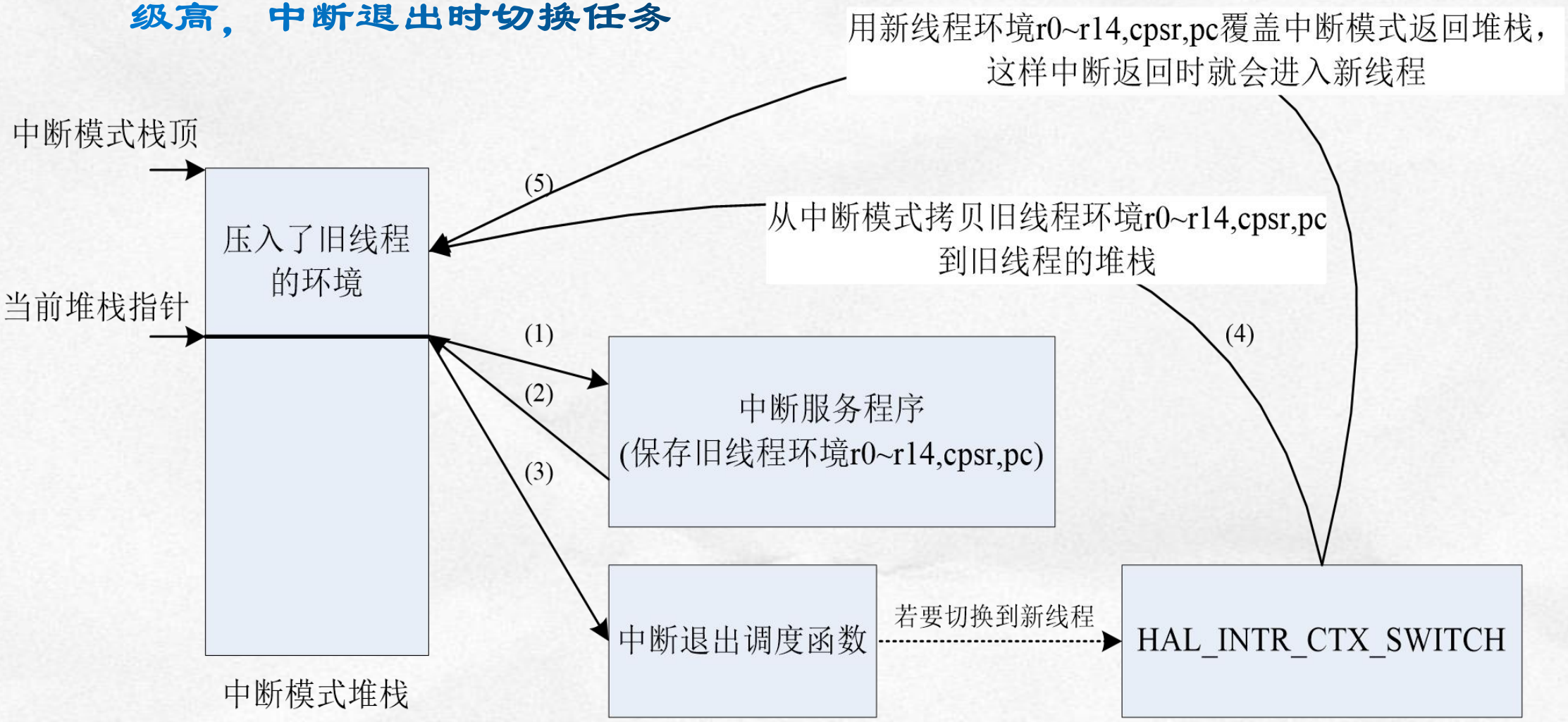
调度任务：任务切换（2）

- **原理图**



调度任务：任务切换（3）

- **中断模式下的任务切换** —— 旧任务先被中断（保存现场）；若新的优先级高，中断退出时切换任务



调度任务：任务退出

- 任务处理完成后的状态　　等待、或者退出
- 退出的方式　　显式、或者隐含

- µcOS的任务及退出

```
// 一般任务
void test(void *ptr){
        Do_something();
            While(1);
}
// 需要退出的任务
void test(void *ptr){
        Do_something();
            EXIT();
}
```

- aCoral的任务及退出

```
// 一般任务
void test(void *ptr){
      Do_something();
}
// 需要退出的任务：执行kill处理
void acoral_thread_exit(){
        acoral_kill_thread(acoral_cur_thread);
}
```

调度任务：任务退出（2）

```
void acoral_kill_thread(acoral_thread_t *thread){
        ......                                  // 定义变量
        ......                                  // 确定是否为当前处理器：多核准备

        HAL_ENTER_CRITICAL();
        // 若在挂起、等待、事件队列上，取下
        if(thread->state&ACORAL_THREAD_STATE_SUSPEND){
                ......
        }
        // 从就绪队列中取下
        acoral_unrdy_thread(thread);

        // 释放任务资源
        acoral_release_thread1(thread);
        HAL_EXIT_CRITICAL();
        acoral_sched();
}
```

调度任务：任务退出 —— 离开就绪

```
void acoral_unrdy_thread(acoral_thread_t *thread){
#ifdef CFG_CMP
        acoral_u32 CPU;
        CPU=thread->CPU;
 //如果线程所在的核不在当前核心上，则通知线程所在的核进行suspend操作
        if(CPU!=acoral_current_CPU){
        acoral_ipi_cmd_send(CPU,ACORAL_IPI_THREAD_SUSPEND,thread);
                return;
        }
#endif
        //将线程从就绪队列中取下
        if(ACORAL_THREAD_STATE_READY==thread->state)
                acoral_rdyqueue_del(thread);
}
```

确定所属
CPU

调度任务：任务退出 —— 离开就绪（2）

```
void acoral_rdyqueue_del(acoral_thread_t *thread){
        acoral_u32 CPU;
        acoral_rdy_queue_t *rdy_queue;
        CPU=thread->CPU;
        //找到所属CPU的就绪队列
        rdy_queue=acoral_ready_queues+CPU;
        //从优先级队列取下
        acoral_prio_queue_del(&rdy_queue->array,thread->prio,&thread->ready);
        thread->state=ACORAL_THREAD_STATE_SUSPEND;
        //设置线程所在的核可调度！！
        acoral_set_need_sched(CPU, true);
}
```

调度任务：任务退出 —— 释放资源

```
void acoral_release_thread1(acoral_thread_t *thread){
        acoral_list_t *head,*tmp;
        acoral_thread_t *daem;
        thread->state=ACORAL_THREAD_STATE_EXIT;
        head=&acoral_res_release_queue.head;                    // 加入回收队列
        acoral_spin_lock(&head->lock);
        tmp=head->prev;
        if(tmp!=head)
                acoral_spin_lock(&tmp->lock);
        acoral_list_add2_tail(&thread->waiting, head);
        if(tmp!=head)
                acoral_spin_unlock(&tmp->lock);
        acoral_spin_unlock(&head->lock);
        daem=(acoral_thread_t *)acoral_get_res_by_id(daemon_id);
        acoral_rdy_thread(daem);
}
```

此状态下
不可回收
一些附加操作

资源回收

挂起任务

- **acoral_suspend_thread()** 挂起，重调度
- **acoral_unrdy_thread()** 挂起，不调度

```
void acoral_suspend_thread(acoral_thread_t *thread){
    ......
    if(!(ACORAL_THREAD_STATE_READY&thread->state))   return;
    #ifdef CFG_CMP
        CPU=thread->CPU;
        if(CPU!=acoral_current_CPU){
            acoral_ipi_cmd_send(CPU,ACORAL_IPI_THREAD_SUSPEND,thread->res.id,NULL);
            return;
        }
    #endif
    HAL_ENTER_CRITICAL();
        acoral_rdyqueue_del(thread);
    HAL_EXIT_CRITICAL();
    acoral_sched();                                    // 重调度
}
```

恢复任务

- **acoral_resume_thread()** 恢复任务到就绪队列，重调度；**用户可用**
- **acoral_rdy_thread()** 恢复任务到就绪队列，不调度；**内部使用**

```
void acoral_rdy_thread(acoral_thread_t *thread){
    ......
    if (!(thread->state&ACORAL_THREAD_STATE_SUSPEND))   return;
    #ifdef CFG_CMP
        CPU=thread->CPU;
        /*resumed*/
        if(CPU!=acoral_current_CPU){
            acoral_ipi_cmd_send(CPU,ACORAL_IPI_THREAD_RESUME,thread->res.id,NULL);
            return;
        }
    #endif
        HAL_ENTER_CRITICAL();
        acoral_rdyqueue_add(thread);
        HAL_EXIT_CRITICAL();
}
```

改变任务优先级

- **起因**　　解决任务优先级反转等问题时需要
- **acoral_thread_change_prio()**

```
void acoral_thread_change_prio(acoral_thread_t* thread, acoral_u32 prio){
    acoral_sr CPU_sr;            // 参数定义
    acoral_u32 CPU;
    ......                                  // 选择处理器
    HAL_ENTER_CRITICAL();
    if (thread->state&ACORAL_THREAD_STATE_READY){
        acoral_rdyqueue_del(thread);
        thread->prio = prio;
        acoral_rdyqueue_add(thread);        // 重新调整就绪队列
    }else
        thread->prio = prio;
    HAL_EXIT_CRITICAL();
}
```

调度策略的时间处理

- **ticks** 心跳 —— 系统时间基数，由晶振精度和用户需求确定
- **acoral_Ticks_entry()** 时钟中断服务程序入口；每ticks一次
- **相关的调度策略** 周期性调度、时间片轮转、EDF等

```
void acoral_Ticks_entry(acoral_vector vector){
    #ifdef CFG_HOOK_TickS
        acoral_Ticks_hook();            // 功能添加接口
    #endif
    Ticks++;                            //
    if(acoral_start_sched==true){
        time_delay_deal();              // 延时处理
        acoral_policy_delay_deal();     // 调度策略相关时间处理
        timeout_delay_deal();           // 超时处理
        .......                         // 用户扩展处理
        }
}
```

时钟管理

调度策略的时间处理（2）

- 策略不同，调用的延迟处理不同
- 策略注册时确定 —— 体现策略与机制分离原则

```
slice_policy_init(){
    slice_policy.type=ACORAL_SCHED_POLICY_SLICE;
    slice_policy.policy_thread_release=slice_policy_thread_release;
    slice_policy.policy_thread_init=slice_policy_thread_init;
    slice_policy.delay_deal=slice_delay_deal;
    slice_policy.name="slice";
    acoral_register_sched_policy(&slice_policy);    // 注册
}
```

册的策略表

```
mp->next){
sched_policy_t,list);
)
        policy_ctrl->delay_deal();          // 对每个策略处理
    }
}
```

调度策略的时间处理（3）

```
void slice_delay_deal(){
    acoral_thread_t *cur;                    // 当前任务
    slice_policy_data_t *data;               // 时间片策略数据结构

    ......                                   // 获取当前任务：当前执行/未执行
    if(cur->policy==ACORAL_SCHED_POLICY_SLICE){
        cur->slice--;                        // 时间片相关任务的时间片值减一
        if(cur->slice<=0){                   // 时间到
            data=(slice_policy_data_t *)cur->private_data;  // 调度策略为时间片
            cur->slice=data->slice_ld;       // 恢复任务的时间片初始值
            acoral_thread_move2_tail(cur);   //当前任务 调度
        }
    ......
}
```

可能有多个任务需要时间片处理

可能同时多个任务时间片到

当前任务的TCB

调度策略的时间处理（4）

```
void acoral_thread_move2_tail(acoral_thread_t *thread){
        acoral_8 CPU;
        acoral_sr CPU_sr;
        ......                                  // 找到特定的CPU
        HAL_ENTER_CRITICAL();
        acoral_unrdy_thread(thread);            // 当前任务移出就绪队列
        acoral_rdy_thread(thread);              // 任务移入就绪队列尾
        HAL_EXIT_CRITICAL();
        acoral_sched();                         // 重调度
}
```

练习题

1、什么是任务的上下文切换？aCoral中，任务上下文切换有哪两种类型？上下文切换的主要内容是什么？请提供aCoral在ARM mini2440上任务切换的代码并进行解释。

2、假定任务优先级分为256级，每个级别对应唯一任务，在aCoral如何利用优先级位图法从就绪队列中找到最高优先级任务？能否做到查找时间和队列长度无关的？

3、什么是调度机制略？什么是调度策略？

4、请给出aCoral创建任务的流程图，并对其TCB初始化进行详细解释。

谢谢！